CodeContests dataset [14]. For the generations, we test the correctness of the solutions via autojudge (see Appendix D.2) and measure the accuracy as n_i^{correct}/k.

D.4 Measuring Runtime Efficiency

For runtime efficiency, we use the bash command `/usr/bin/time` [52] to measure the elapsed real time (via %E) as well as the maximum resident set size of the process during its execution (via %M). Concretely, for each solution, we run it on all 10 test cases with our autojudge and measure the runtime and memory. We take the max value on all 10 test cases as a proxy of the runtime efficiency of the tested code.

D.5 Prompting `GPT-3.5` to Generate Code Summary (both Text Descriptions and Categorical Values)

We present below the instruction we provide to `GPT-3.5`. We bold keywords in the prompt simply for the ease of reading. When calling the `GPT-3.5` API, we used temperature $T = 0$ (i.e., greedy decoding) and `max_tokens=500`.

The list of tags (in the 1st prompt below) were collected from the CodeContests [14] dataset—we traverse all the problems in the dataset and union all the "tags" attribute.

The lists of algorithms and data structures (in the 2nd prompt below) were obtained from analyzing Wikipedia and querying `GPT-4-turbo (0125)` for a suggested list.

These categorical attributes serve as a complement to the text description described above and enhance the reliability of the results.

> Please provide a description to the following code in natural language. Explain the **functionality, algorithm, data structure, time complexity, space complexity** of the code.
> Finally, assign a few **tags** to the code. Here is a list of tags you can choose from:
> "binary search, math, special, trees, dp, greedy, games, dfs and similar, expression parsing, number theory, chinese remainder theorem, geometry, bitmasks, sortings, graph matchings, matrices, meet-in-the-middle, graphs, combinatorics, probabilities, constructive algorithms, schedules, two pointers, brute force, dsu, shortest paths, hashing, interactive, data structures, strings, ternary search, fft, flows, implementation"
> Answer each in a line in the example format of: 'Description: description\nFunctionality: functionality'
> {`code`}

> Please read the following code and infer the **algorithms** and **data structures** used in it.
> For algorithms, select (a few) from the following list:
> "Sorting Algorithms, Searching Algorithms, String Algorithms, Divide and Conquer Algorithms, Greedy Algorithms, Dynamic Programming, Recursion, Bit Manipulation, Backtracking, Graph Algorithms, Others"
> For data structures, select (a few) from the following list:
> "Arrays, Linked Lists, Stacks, Queues, Trees, Heaps, Hash Tables, Sets, Maps, Priority Queues, Others"
> Answer each in a line following the format of: 'Algorithms: candidate 1, candidiate 2, ..\nData structures: candidate 1, candidiate 2, ..\n'
> {`code`}

E Additional Results: Coding

E.1 Examples for Plagiarism Scores

We present example model-generated code and human-written code as well as the plagiarism scores associated with the pairs in Fig. 14, 15,and 16. All the code are correct solutions to the problem 409A. The Great Game[5].

From the demonstrations it is evident that model-generated code (Fig. 14, 15) are highly similar in their style and structure; even the low score pair (Fig. 15) bear a significant level of similarity. In comparison, human-written code (Fig. 16) are clearly distinctive. These results support the validity of using the plagiarism score to evaluate code similarity.

[5]Link to the problem: `https://codeforces.com/problemset/problem/409/A`

Test file: *train_gen/train-00000_0/train-00000_solution_codeonly_5_0.py* (**78.00%**)
Reference file: *train_gen/train-00000_0/train-00000_solution_codeonly_14_0.py* (**79.19%**)
Token overlap: 156

View matched code

```python
n = int(input())
last_year = [input() for _ in range(n)]
current_year = [input() for _ in range(n)]

last_year_count = {}
current_year_count = {}

for size in last_year:
    if size in last_year_count:
        last_year_count[size] += 1
    else:
        last_year_count[size] = 1

for size in current_year:
    if size in current_year_count:
        current_year_count[size] += 1
    else:
        current_year_count[size] = 1

seconds = 0
for size in current_year_count:
    if size in last_year_count:
        if current_year_count[size] > last_year_count[size]:
            seconds += current_year_count[size] - last_year_count[si
    else:
        seconds += current_year_count[size]

print(seconds)
```

```python
n = int(input())
prev_year = [input() for _ in range(n)]
current_year = [input() for _ in range(n)]

prev_count = {}
current_count = {}

for size in prev_year:
    if size in prev_count:
        prev_count[size] += 1
    else:
        prev_count[size] = 1

for size in current_year:
    if size in current_count:
        current_count[size] += 1
    else:
        current_count[size] = 1

seconds = 0
for size in current_count:
    if size in prev_count:
        diff = current_count[size] - prev_count[size]
        if diff > 0:
            seconds += diff
    else:
        seconds += current_count[size]

print(seconds)
```

Figure 14: An example for **a pair of model generated code** with **high plagiarism score**.

Test file: *train_gen/train-00000_19/train-00000_solution_codeonly_15_19.py* (**33.64%**)
Reference file: *train_gen/train-00000_19/train-00000_solution_codeonly_7_19.py* (**23.00%**)
Token overlap: 72

```python
def find_winner(team1_actions, team2_actions):
    scores = {"8<": 0, "[]": 1, "()": 2}
    team1_score = 0
    team2_score = 0

    for i in range(0, len(team1_actions), 2):
        action1 = team1_actions[i:i+2]
        action2 = team2_actions[i:i+2]

        if scores[action1] == (scores[action2] + 1) % 3:
            team1_score += 1
        elif scores[action2] == (scores[action1] + 1) % 3:
            team2_score += 1

    if team1_score > team2_score:
        return "TEAM 1 WINS"
    elif team2_score > team1_score:
        return "TEAM 2 WINS"
    else:
        return "TIE"

team1_actions = input().strip()
team2_actions = input().strip()

print(find_winner(team1_actions, team2_actions))
```

```python
def determine_winner(actions_team1, actions_team2):
    score_team1, score_team2 = 0, 0
    # Points system based on action pairs
    points_system = {
        ('8<', '[]'): 1, ('[]', '8<'): 1,
        ('()', '8<'): 1, ('8<', '()'): 1,
        ('[]', '()'): 1, ('()', '[]'): 1
    }

    # Split the actions into 2-character pairs
    actions_team1 = [actions_team1[i:i+2] for i in range(0, len(actio
    actions_team2 = [actions_team2[i:i+2] for i in range(0, len(actio

    # Compare actions and calculate scores
    for action1, action2 in zip(actions_team1, actions_team2):
        if (action1, action2) in points_system:
            score_team1 += points_system[(action1, action2)]
        elif (action2, action1) in points_system:
            score_team2 += points_system[(action2, action1)]

    # Determine the winner based on the scores
    if score_team1 > score_team2:
        return "TEAM 1 WINS"
    elif score_team2 > score_team1:
        return "TEAM 2 WINS"
    else:
        return "TIE"

# Reading input
actions_team1 = input().strip()
actions_team2 = input().strip()

# Determining the winner
result = determine_winner(actions_team1, actions_team2)
print(result)
```

Figure 15: An example for **a pair of model generated code** with **relatively low plagiarism score**.

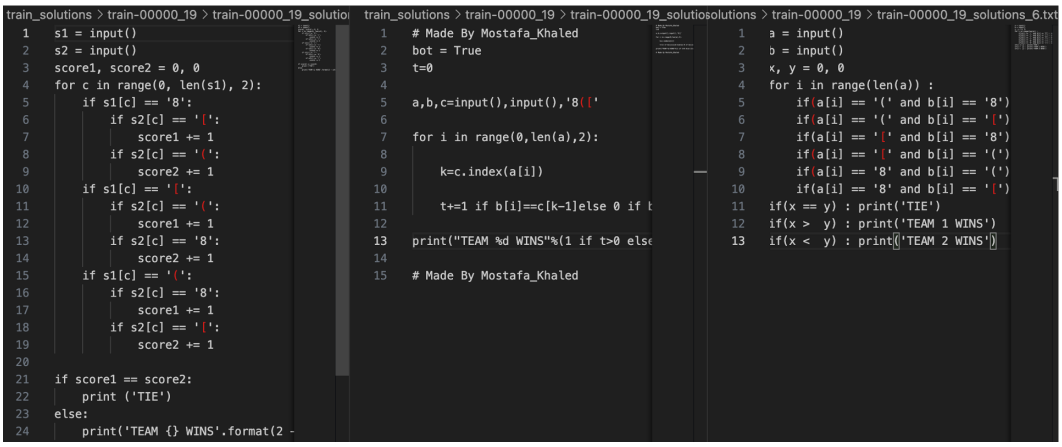

Figure 16: An example for **three human-written code** with ***zero*** **plagiarism score** for all the pairs.

E.2 Attempts on Varying Prompts

We experimented with four prompts on GPT-4, one plain prompt, one instructing the model to generate only the code solution, one employing role-playing [31] and one integrating chain-of-thought prompting [53]. We present the concrete prompts in Fig. 17.

We evaluate the four prompts on Codeforces problems of varying difficulties and present the accuracy achieved by each prompt in Fig. 18. Overall, prompt 2 achieves the best results among the four.

E.3 Human annotations for the quality of LLM summary

We annotate the quality of the LLM-generated summary of code given by `GPT-3.5`.

Step 1: We randomly sample 20 solutions of different problems, and review the LLM-generated attributes for these solutions; the attributes include the textual ones (description, functionality, algorithm, data structure), the inferred asymptotic complexity (time complexity, space complexity), as well as the categorical ones (tags, algorithm, data structures).

Step 2: We read through the problem description, the code solution, and the model-generated summary to evaluate the quality of model-generated summary.

Step 3: We follow the instruction "In each cell please give a score from 1-3 – 3 meaning absolutely correct, 2 meaning with some minor errors but acceptable, 1 meaning entirely incorrect" to score each attribute.

Step 4: We calculate the average score received by each attribute across the 20 sample and present the results in Table 3.

The high scores indicate that `GPT-3.5` can provide fairly accurate summary of the given code, supporting our choice of relying `GPT-3.5` to reliably extract the attributes.

Table 3: **The average scores given by human annotators on each attribute.** We adopted a 3-point scale where 3 means absolutely correct, 2 means with some minor errors but acceptable, 1 means entirely incorrect. The high scores indicate that `GPT-3.5` can provide fairly accurate summary of the given code.

Attribute	textual				complexity		categorical		
	description	functionality	algorithm	data structure	time complexity	space complexity	tags	algorithms	data structures
Score	2.65	2.55	2.90	3.00	2.95	2.70	2.90	2.50	2.90